

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação
Especialização em Gerência de Projetos de Software na Era de Dados de
Sensores e IA

Detecção de Queda na Indústria 4.0

Camila Corrêa Vieira

Professor: Marco Antônio Pereira Araújo

Trabalho final de Engenharia de Software Moderna, parte integrante da avaliação da disciplina.

Juiz de Fora

01 de Setembro de 2025

1 Introdução

O avanço da Indústria 4.0, com o aumento do uso de tecnologias digitais e automação em setores como fábricas e logística, traz novos e importantes desafios para a segurança no trabalho. Um dos maiores riscos está na situação de trabalhadores que atuam sozinhos em grandes áreas industriais. Nesses locais, um acidente como uma queda pode não ser notado imediatamente, criando uma demora perigosa entre o incidente e a chegada do socorro, o que pode agravar muito as consequências para a pessoa.

Para resolver esse problema de segurança, este trabalho apresenta a proposta de criação de um Sistema de Monitoramento e Resposta a Emergências para o Trabalhador Conectado. É uma solução de software inteligente que funciona em tempo real. O sistema utilizará dispositivos vestíveis (smartwatches) com sensores de movimento (acelerômetro e giroscópio) para coletar continuamente dados sobre a movimentação do trabalhador. Esses dados são enviados para uma plataforma central, onde um modelo de Inteligência Artificial analisa os padrões para identificar eventos de queda, diferenciando-os de movimentos rápidos e normais do dia a dia.

A base da arquitetura do sistema utiliza o padrão de microserviços. Essa foi uma escolha estratégica para garantir que o sistema possa crescer facilmente (escalabilidade), atendendo desde uma única fábrica em testes até milhares de trabalhadores, e também para ser mais confiável (resiliência), pois a falha de um componente não paralisa a função crítica de alerta. Todo o processo de desenvolvimento e atualização do software é apoiado por uma cultura e ferramentas de DevSecOps, que automatizam a entrega de novas versões e incluem a segurança em todas as etapas, permitindo que o sistema evolua de forma rápida e segura.

O objetivo principal, portanto, é projetar um sistema de alta importância que reduza drasticamente o tempo de resposta a acidentes, transforme dados brutos em informações úteis para a segurança e crie uma nova forma de cuidar dos trabalhadores de maneira antecipada (proativa). Ao longo deste relatório, serão apresentados os requisitos do projeto, a arquitetura detalhada, os padrões utilizados, as estratégias de teste e automação, e uma discussão final sobre o potencial e as limitações do sistema.

2 Mapeamento de Requisitos Ágeis

2.1 Personas

- Júlio, o Operador de Manutenção: Representa o usuário final do dispositivo IoT, cujo principal objetivo é realizar suas tarefas com a garantia de uma rede

de segurança automatizada que não interfira em seu trabalho. Ele é o principal beneficiário da funcionalidade de detecção de queda.

- Sônia, a Supervisora de Segurança: Personifica a usuária da plataforma de monitoramento (dashboard), focada na gestão da segurança da equipe, na obtenção de consciência situacional em tempo real e na eficiência da coordenação da resposta a incidentes.

2.2 Histórias de Usuário (User Stories)

As funcionalidades do sistema foram capturadas na forma de histórias de usuário, seguindo o formato "Como um <ator>, eu quero <ação>, para que <objetivo>". Esta técnica assegura que cada requisito seja rastreável a um valor tangível para uma persona. Os exemplos mais críticos que guiaram a definição da arquitetura central são:

"Como Júlio, quero que uma queda seja detectada automaticamente, para que o socorro seja chamado mesmo se eu estiver inconsciente."

"Como Sônia, quero receber um alerta instantâneo com a localização exata de um trabalhador acidentado, para despachar a equipe de socorro para o local certo imediatamente."

2.3 Produto Mínimo Viável (MVP)

A estratégia de desenvolvimento prioriza a validação da hipótese central do produto, a viabilidade de detectar uma queda e notificar um responsável de forma automatizada, com o mínimo de esforço de desenvolvimento. O escopo do MVP está focado em comprovar o fluxo de negócio mais crítico:

- Funcionalidade Central: O sistema deverá ser capaz de processar um fluxo de dados simulado de um acelerômetro, aplicar uma regra heurística simples para a detecção de queda (por exemplo, aceleração vetorial superior a 3g) e disparar uma notificação por e-mail para um endereço pré-configurado.
- Objetivo Estratégico: Validar a viabilidade técnica do fluxo de dados *Sensor -> Processamento Lógico -> Alerta* de ponta a ponta, servindo como base para iterações futuras.

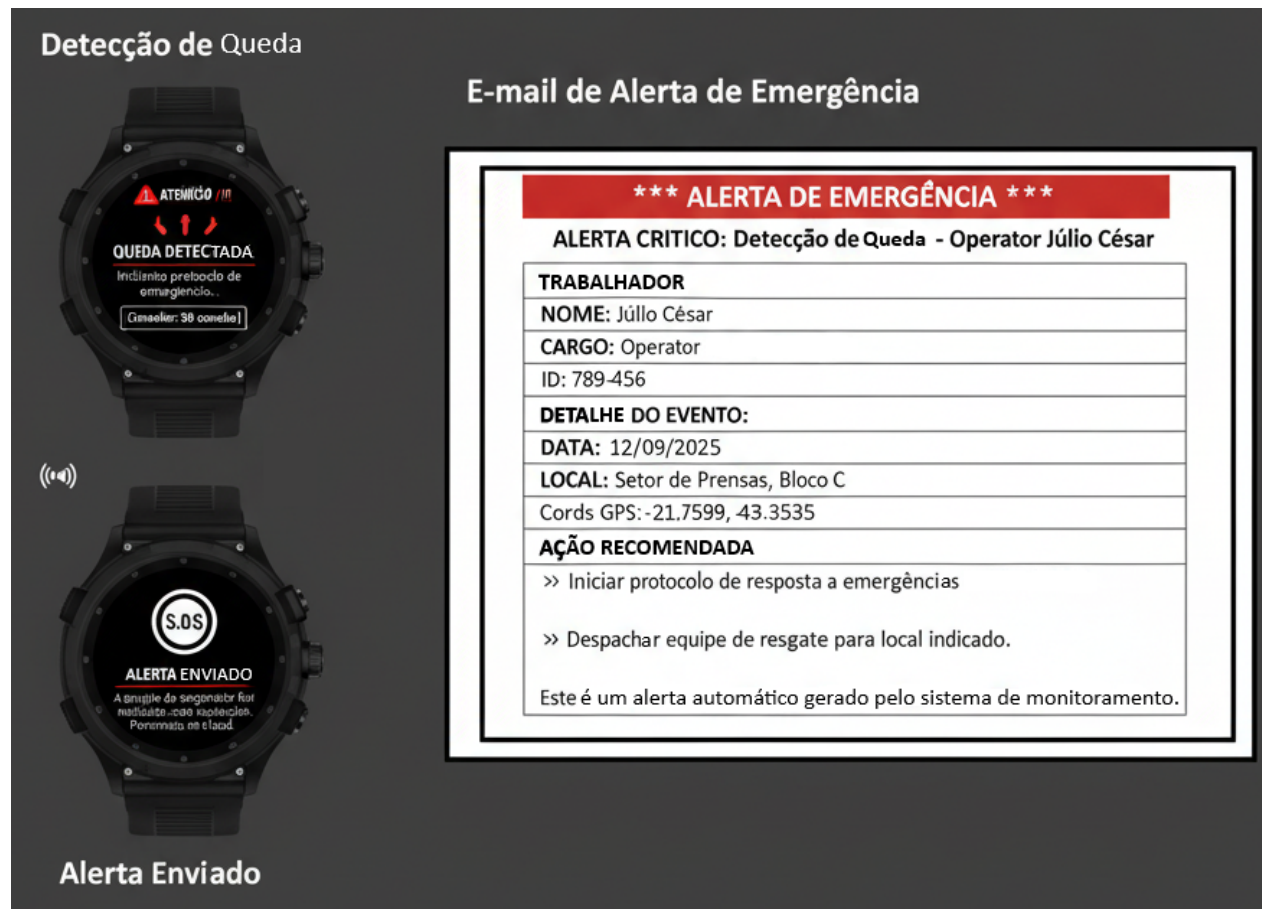


Figura 1: Protótipo de tela básico para o MVP

2.4 Estratégia de Experimentação

A evolução do produto para além do MVP será guiada por uma abordagem de experimentação contínua. Um dos primeiros experimentos planeados visa otimizar a interface de alerta para a supervisora (persona Sônia), utilizando um teste A/B.

- Hipótese: Uma interface de alerta enriquecida, contendo um mapa visual da localização do incidente, resultará em um tempo de confirmação e resposta mais rápido por parte da supervisora em comparação com um alerta puramente textual.
- Grupos de Teste:
 - *Grupo A*: Receberá um alerta de texto simples.
 - *Grupo B*: Receberá um alerta contendo texto e um mapa interativo.
- Métrica Chave de Sucesso (KPI): Tempo médio (em segundos) entre a exibição do alerta na tela da supervisora e a sua ação de confirmação ("ciente" ou "despachar equipe").

3 Arquitetura de Software Proposta

3.1 Diagrama de Componentes e Interação (Arquitetura de Microsserviços):

A arquitetura de microsserviços do sistema é composta por um conjunto de serviços especializados, de baixo acoplamento e alta coesão, que interagem através de APIs bem definidas para compor o fluxo de detecção e resposta a emergências. Os componentes principais são descritos a seguir:

1. Smartwatch (Dispositivo IoT) Representando a "borda"(edge) do sistema, o smartwatch é o dispositivo físico responsável pela coleta contínua de dados cinéticos através de seus sensores embarcados (acelerômetro, giroscópio) e dados de localização (GPS). Sua função é pré-processar minimamente esses dados e transmiti-los de forma segura e periódica para o backend através de uma conexão de rede.
2. API Gateway Atua como a única porta de entrada (single entry point) para o ecossistema de serviços. Este componente é crucial para a segurança e gerenciamento do sistema, sendo responsável por tarefas como: Autenticação e Autorização: Validar a identidade do dispositivo e garantir que ele tem permissão para enviar dados. Roteamento de Requisições: Direcionar as requisições recebidas para o microsserviço

apropriado (neste caso, o Serviço de Ingestão de Dados). Rate Limiting: Proteger os serviços internos contra sobrecargas ou ataques.

3. Serviço de Ingestão de Dados (Data Ingestion Service) Este serviço é a linha de frente para o recebimento de dados. Sua principal responsabilidade é consumir os fluxos de dados enviados pelos smartwatches de forma altamente escalável e assíncrona. Ele valida, normaliza e enriquece os dados brutos (ex: adicionando um timestamp do servidor) e, em seguida, os publica em um tópico de um message broker (como Apache Kafka ou RabbitMQ), desacoplando o processo de coleta do processo de análise.
4. Serviço de Detecção de Queda (IA) (Fall Detection Service) Considerado o núcleo de inteligência do sistema, este serviço consome os dados de sensores do message broker. Ele aplica um modelo de Inteligência Artificial (ex: uma Rede Neural Recorrente - RNN/LSTM) para analisar sequências de dados em uma janela de tempo e classificar padrões de movimento como "Queda" ou "Não Queda". Ao identificar um evento de queda com alta probabilidade, ele não executa a lógica de negócio, mas sim publica um evento de domínio, como `EventoDeQuedaDetectada`, em um novo tópico, mantendo sua responsabilidade única.
5. Serviço de Protocolo de Emergência (Emergency Protocol Service) Este serviço atua como o orquestrador da lógica de negócio de emergência. Ele escuta os eventos `EventoDeQuedaDetectada` e inicia um fluxo de trabalho (workflow) sensível ao contexto. Suas etapas incluem:
 - Iniciar um temporizador (ex: 30 segundos).
 - Enviar uma notificação de "confirmação de bem-estar" de volta ao smartwatch do usuário.
 - Aguardar uma resposta do usuário ou o término do temporizador.
 - Com base no resultado (confirmação do usuário, ausência de resposta ou pedido de ajuda), ele publica um evento final, como `AlertaDeEmergenciaConfirmado` ou `AlarmeFalsoRegistrado`.
6. Serviço de Notificação (Notification Service) Um serviço genérico e altamente reutilizável que escuta por eventos como `AlertaDeEmergenciaConfirmado`. Sua única função é despachar as notificações para os canais apropriados, integrando-se com serviços de terceiros. Por exemplo, ele pode enviar um SMS (via Twilio), um e-mail (via SendGrid) e uma mensagem em tempo real (via WebSockets) para o dashboard.
7. Serviço de Dashboard (Dashboard Service) Representa a interface homem-máquina (HMI) do sistema, destinada à supervisora Sônia. É uma aplicação web, possível-

mente desenvolvida com o padrão MVC ou uma arquitetura de SPA (Single Page Application), que: Recebe alertas e atualizações de status em tempo real (escutando eventos ou via WebSockets). Exibe a localização e o status de todos os trabalhadores em um mapa interativo. Permite a visualização de históricos de eventos e a geração de relatórios.

A orquestração de um evento de emergência segue um fluxo de dados sequencial e orientado a eventos através da arquitetura de microsserviços. O processo é iniciado na borda pelo Smartwatch, que transmite os dados cinéticos através do API Gateway para o Serviço de Ingestão. Este, por sua vez, publica os dados de forma assíncrona em um message broker, garantindo o desacoplamento e a resiliência do sistema. Subsequentemente, o Serviço de Detecção (IA) consome esses dados, aplicando um modelo de inferência para identificar padrões de queda e, em caso positivo, publica um `EventoDeQuedaDetectada`. A partir deste ponto, o Serviço de Protocolo de Emergência é acionado para orquestrar a lógica de confirmação e, se necessário, emitir um `AlertaDeEmergenciaConfirmado`. Finalmente, este alerta é consumido pelo Serviço de Notificação, que realiza a disseminação da informação para os canais relevantes, culminando na atualização em tempo real do Dashboard de monitoramento. Todo o ciclo de detecção-resposta é projetado para operar com baixa latência, concluindo-se em questão de segundos.

3.2 Justificativas para Escolhas Arquiteturais:

A adoção da arquitetura de microsserviços para este sistema é uma decisão estratégica fundamentada em três pilares essenciais, que se alinham diretamente aos requisitos não funcionais do projeto [7, 8]:

- **Escalabilidade:** A arquitetura permite a escalabilidade horizontal e seletiva dos componentes. Em vez de escalar a aplicação monolítica inteira, é possível alocar recursos de forma independente para os serviços de maior demanda, como o Serviço de Ingestão de Dados e o Serviço de Detecção de Queda. Isso garante que o sistema possa suportar um aumento exponencial no número de trabalhadores monitorados de forma custo-efetiva.
- **Resiliência e Isolamento de Falhas (Fault Isolation):** A natureza distribuída dos microsserviços garante que a falha de um componente não-crítico não comprometa a funcionalidade vital do sistema. Por exemplo, uma indisponibilidade temporária no Serviço de Dashboard, responsável pela visualização, não impede que o fluxo principal de detecção de quedas e envio de alertas de emergência continue operando, assegurando a alta disponibilidade da função de segurança primária.

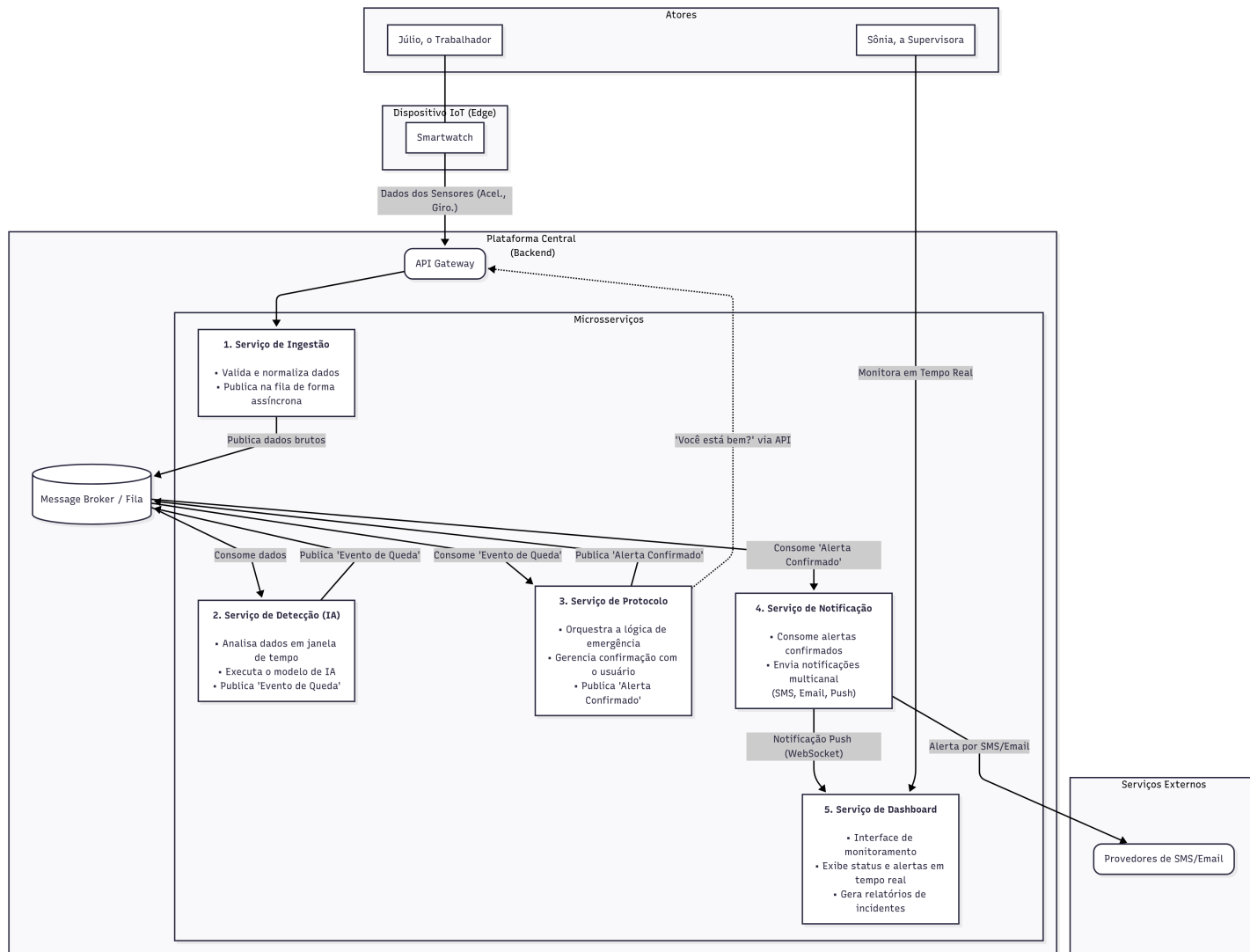


Figura 2: Diagrama que mostra as principais responsabilidades de cada microserviço

- **Manutenibilidade e Agilidade na Evolução:** O baixo acoplamento entre os serviços permite que equipes de desenvolvimento trabalhem de forma autônoma e realizem implantações independentes. Isso acelera o ciclo de vida do desenvolvimento e facilita a evolução contínua do sistema. Por exemplo, o modelo de Inteligência Artificial no Serviço de Detecção pode ser atualizado, testado e implantado sem qualquer impacto ou necessidade de reimplantação dos demais serviços da plataforma.

4 Padrões e Princípios de Projeto Aplicados

A concepção arquitetônica do sistema é fortemente influenciada por padrões de projeto e princípios que garantem sua resiliência e manutenibilidade a longo prazo. No nível da comunicação entre serviços, a arquitetura emprega o padrão Publish-Subscribe, implementado pelo Message Broker. Este mecanismo, que funciona como uma implementação distribuída do padrão Observer, permite que serviços como o *Serviço de Protocolo de Emergência* publiquem eventos de domínio de forma assíncrona, sem qualquer acoplamento com os serviços consumidores, como o *Serviço de Notificação*, que subscrevem a estes eventos para executar suas respectivas lógicas [2]. O acesso ao ecossistema de serviços é gerenciado pelo padrão API Gateway, que serve como uma fachada centralizada. Ele abstrai a complexidade interna da arquitetura para os clientes externos e lida com responsabilidades transversais essenciais, como segurança, roteamento e monitoramento. Internamente, cada microsserviço é desenvolvido sob a égide dos princípios SOLID. O Princípio da Responsabilidade Única assegura que cada serviço tenha um propósito coeso e bem definido, por exemplo, o *Serviço de Notificação* é exclusivamente responsável pelo despacho de alertas. Consequentemente, o Princípio Aberto/Fechado guia o design para que a plataforma seja extensível, permitindo que novos canais de notificação sejam adicionados sem alterar o código existente, promovendo assim a evolução contínua e segura do sistema [6].

5 Estratégia de Evolução e Manutenção Contínua

O sistema foi concebido não como um produto estático, mas como uma plataforma em constante evolução. Para garantir sua sustentabilidade, longevidade e alinhamento contínuo com as melhores práticas tecnológicas, adota-se uma estratégia proativa de evolução, fundamentada em duas atividades centrais: o gerenciamento sistemático da dívida técnica e a refatoração arquitetural planejada.

5.1 Gerenciamento Sistemático da Dívida Técnica

A dívida técnica, inerente a todo processo de desenvolvimento de software [1], será gerenciada de forma explícita e sistemática, em vez de reativa. Para tal, um backlog de dívida técnica será mantido como um artefato formal do projeto. Neste backlog, débitos técnicos, como a necessidade de refatoração de código, otimização de algoritmos ou a ampliação da cobertura de testes, serão catalogados, priorizados e estimados. A mitigação dessa dívida será integrada ao ciclo de desenvolvimento através da alocação de uma porcentagem da capacidade de cada iteração (sprint) ou pela execução de ciclos de refatoração planejada em intervalos regulares (por exemplo, a cada quatro sprints). O objetivo desta prática é preservar a qualidade interna do código (saúde do código), a manutenibilidade e a agilidade da equipe a longo prazo.

5.2 Evolução Arquitetural e Refatoração Planejada

Além da gestão da dívida no nível do código, a própria arquitetura do sistema é projetada para evoluir. Componentes-chave foram identificados desde a concepção para futuras otimizações, permitindo que o sistema amadureça de forma controlada e eficiente. Os principais pontos de refatoração previstos são:

- **Evolução do Modelo de Inferência (IA):** A versão inicial do sistema (MVP) empregará uma implementação de detecção baseada em heurísticas ou um modelo estatístico simples, visando acelerar a entrega de valor. Contudo, uma refatoração arquitetural planejada consiste na substituição deste componente por um modelo de Machine Learning mais sofisticado, como uma Rede Neural Recorrente (LSTM), treinado em um dataset mais abrangente. Esta evolução visa um aumento significativo na acurácia da detecção e uma consequente redução na taxa de falsos positivos.
- **Otimização do Protocolo de Comunicação IoT:** A comunicação inicial entre os dispositivos vestíveis e o backend será implementada utilizando o protocolo HTTP/S, priorizando a simplicidade de implementação e a ubiquidade da tecnologia. Estrategicamente, está prevista uma evolução para um protocolo otimizado para o ecossistema IoT, como o MQTT (Message Queuing Telemetry Transport). Esta migração visa ganhos substanciais em eficiência energética, fator crucial para a autonomia da bateria dos dispositivos, e a redução do consumo de banda de rede, tornando a solução mais escalável e robusta em ambientes com conectividade limitada [4].

6 Plano de Testes

A garantia de qualidade do sistema é um pilar fundamental do projeto, dada a sua natureza crítica. Adota-se uma estratégia de verificação e validação (V&V) proativa e automatizada, fundamentada em uma abordagem estratificada de testes e na metodologia de Desenvolvimento Orientado a Testes (TDD), visando assegurar a robustez, confiabilidade e correteza do software em todos os seus níveis.

6.1 Abordagem Estratificada de Testes Automatizados (Pirâmide de Testes)

A estratégia de automação de testes segue o modelo da Pirâmide de Testes, que preconiza um grande volume de testes rápidos na base e um número menor de testes mais lentos e abrangentes no topo.

- Testes de Unidade (Base da Pirâmide): No nível fundamental, os Testes de Unidade garantirão o comportamento correto dos componentes isolados (classes, métodos) de cada microsserviço. O escopo inclui a validação de algoritmos críticos, como o cálculo da força G a partir de dados brutos do acelerômetro, e a lógica de negócio interna de cada serviço. A meta é atingir uma cobertura de código (*code coverage*) superior a 90%, metrificada por ferramentas de análise estática, para assegurar alta confiabilidade e facilitar refatorações seguras.
- Testes de Integração (Meio da Pirâmide): Neste nível intermediário, os Testes de Integração validarão a colaboração entre os componentes e serviços. O foco é verificar as "costuras" da arquitetura, incluindo a interação com o Message Broker (garantindo que um serviço consegue serializar e publicar uma mensagem que outro serviço consegue consumir e desserializar corretamente) e a comunicação com bancos de dados. Serão empregados testes de contrato (*contract testing*) para garantir que as APIs entre os serviços permaneçam consistentes e não quebrem as integrações durante a evolução independente de cada microsserviço.
- Testes de Ponta-a-Ponta (E2E - Topo da Pirâmide): No topo da pirâmide, os Testes E2E simularão jornadas completas do usuário através do sistema. Scripts automatizados irão emular o envio de dados de um dispositivo IoT, percorrendo todo o fluxo de microsserviços em um ambiente de testes instrumentado, e verificarão o resultado final esperado, como o recebimento de uma notificação por e-mail contendo as informações corretas do alerta. Estes testes validam o fluxo de negócio de ponta a ponta.

6.2 Metodologia de Desenvolvimento Orientado a Testes (TDD)

O desenvolvimento dos componentes críticos do sistema, especialmente a lógica de detecção e de protocolo de emergência, será guiado pela metodologia TDD. Esta abordagem não é apenas uma técnica de teste, mas uma metodologia de design que assegura que o código seja testável por construção e que os requisitos sejam traduzidos em especificações executáveis (os próprios testes). O ciclo de TDD será seguido rigorosamente:

1. Fase Vermelha (Red): Primeiramente, escreve-se um teste de unidade que falha para uma nova funcionalidade. Este teste define formalmente o comportamento esperado. Por exemplo, um teste para a função `is_fall(sensor_data)` seria criado com uma série temporal de dados que representa uma queda, esperando um retorno `True`.
2. Fase Verde (Green): Em seguida, escreve-se a quantidade mínima de código de produção necessária para que o teste passe, validando a correção funcional da implementação.
3. Fase de Refatoração (Refactor): Finalmente, com a garantia dos testes, o código de produção é refatorado para melhorar sua estrutura interna, legibilidade e desempenho, sem alterar seu comportamento externo validado.

7 Estratégia e Infraestrutura de DevSecOps

Para garantir a entrega de software de forma rápida, confiável e segura, o projeto adota uma cultura e uma infraestrutura de DevSecOps. A abordagem visa integrar as práticas de segurança em todas as fases do ciclo de vida de desenvolvimento de software ("shifting security left"), automatizando a construção, os testes e a implantação de forma contínua [3, 5].

7.1 Ecossistema Tecnológico de Suporte

A estratégia de DevSecOps é suportada por um ecossistema de ferramentas integradas, selecionadas para prover automação e visibilidade em todo o fluxo de entrega:

- Controle de Versão e CI/CD: Git com GitLab e GitLab CI/CD. Uma plataforma unificada que gerencia o repositório de código-fonte e orquestra a automação dos pipelines de integração e entrega contínua.

- Containerização e Orquestração: Docker e Kubernetes. O Docker é utilizado para empacotar cada microsserviço e suas dependências em contêineres padronizados, garantindo a consistência entre os ambientes. O Kubernetes orquestra a implantação, o escalonamento e o gerenciamento destes contêineres em escala.
- Segurança da Aplicação (Sec): SonarQube. Integrado ao pipeline de CI, realiza a Análise Estática de Segurança da Aplicação (SAST), identificando vulnerabilidades, bugs e code smells a cada commit, antes que o código seja mesclado à base principal.
- Observabilidade: Prometheus e Grafana. Um conjunto de ferramentas para a coleta de métricas de desempenho e saúde do sistema em tempo real (Prometheus) e para a criação de dashboards de visualização (Grafana), permitindo o monitoramento proativo e a rápida identificação de anomalias em produção.

7.2 Descrição do Pipeline de CI/CD

O fluxo de entrega de software é dividido em dois estágios principais: Integração Contínua (CI), acionado a cada commit em uma ramificação de funcionalidade (feature branch), e Entrega Contínua (CD), acionado a cada merge na ramificação principal (main branch).

7.2.1 Pipeline de Integração Contínua (CI)

Iniciação: Um desenvolvedor envia um commit para sua feature branch e abre um Merge Request.

Build e Análise Estática: O pipeline do GitLab CI é acionado automaticamente, realizando a construção da imagem Docker do microsserviço e, em paralelo, a análise estática de código e segurança com o SonarQube.

Execução de Testes: Os testes de unidade e de integração automatizados são executados para validar a corretude e a compatibilidade do novo código.

Feedback Rápido: O resultado de todas as etapas é reportado diretamente no Merge Request. A mesclagem do código é bloqueada caso qualquer uma das etapas falhe, garantindo que apenas código validado e seguro seja integrado à base principal.

7.2.2 Pipeline de Entrega Contínua (CD)

1. Publicação do Artefato: Após a aprovação e merge do código na main branch, uma nova imagem Docker é versionada e publicada no GitLab Container Registry.

2. Deploy em Homologação (Staging): O Kubernetes é acionado para implantar a nova versão no ambiente de homologação, uma réplica fidedigna do ambiente de produção.
3. Validação Integrada:** Testes automatizados de ponta-a-ponta (E2E) são executados contra o ambiente de homologação para validar o fluxo de negócio completo e as integrações entre os serviços.
4. Portão de Aprovação Manual: Antes da implantação final, há um "portão" de aprovação manual. Esta etapa permite que a equipe de QA ou o Product Owner realize uma validação final, se necessário, antes de promover a versão para os usuários finais.
5. Deploy em Produção (Blue-Green): Após a aprovação, o pipeline orquestra a implantação em produção utilizando a estratégia Blue-Green. A nova versão (Green) é implantada ao lado da versão estável existente (Blue). Uma vez que a nova versão esteja totalmente operacional e saudável, o tráfego é redirecionado do ambiente Blue para o Green. Esta abordagem mitiga os riscos de implantação, garante zero downtime durante as atualizações e permite um rollback quase instantâneo, simplesmente redirecionando o tráfego de volta para a versão Blue em caso de problemas.

8 Conclusões

O projeto conceitual e técnico aqui apresentado detalha a arquitetura de um sistema ciberfísico de missão crítica, projetado para aumentar a segurança do trabalhador em ambientes da Indústria 4.0. Esta seção final realiza uma análise crítica do sistema, ponderando seus benefícios potenciais, os riscos e limitações inerentes, e a sustentabilidade de sua arquitetura a longo prazo.

8.1 Impacto e Benefícios Potenciais

A implementação bem-sucedida deste sistema tem o potencial de gerar valor em múltiplas frentes. O benefício mais imediato e significativo é o aumento da segurança e bem-estar do trabalhador, através da redução substancial do tempo de resposta a acidentes, o que pode mitigar a gravidade de lesões. Além do impacto humano direto, o sistema transforma dados brutos de sensores em inteligência acionável. A análise contínua desses dados permite a identificação proativa de áreas, horários ou procedimentos de maior risco na planta, fornecendo uma base empírica para a melhoria contínua dos processos de segurança. Por fim, a automação e o registro de incidentes asseguram a conformidade com rigorosas normas de segurança ocupacional, fortalecendo a governança corporativa.

8.2 Riscos, Limitações e Estratégias de Mitigação

Apesar dos benefícios, um sistema desta natureza apresenta desafios e riscos que devem ser gerenciados de forma proativa:

- **Acurácia do Modelo de Inferência:** A eficácia do sistema é diretamente dependente da precisão do modelo de IA em distinguir quedas reais de movimentos bruscos (falsos positivos) e em não falhar ao detectar um evento real (falsos negativos). A principal estratégia de mitigação envolve um ciclo de MLOps (Machine Learning Operations): retreinamento periódico do modelo com novos dados, monitoramento de seu desempenho para detectar drift (degradação do modelo) e a implementação de um feedback loop, onde os supervisores possam classificar os alertas, gerando dados para o reajuste futuro do modelo.
- **Questões Éticas e de Privacidade:** O monitoramento contínuo de indivíduos suscita considerações de privacidade. A mitigação deste risco é tanto técnica quanto política, envolvendo a implementação de políticas de uso de dados transparentes e consentidas, a anonimização de dados para análises estatísticas, e a garantia de que o monitoramento se restrinja estritamente ao perímetro e horário de trabalho, conforme as normas de segurança da empresa.
- **Dependência da Infraestrutura de Rede:** A comunicação em tempo real é vital. Falhas na conectividade (Wi-Fi/celular) representam um ponto de vulnerabilidade. A mitigação inclui o design de uma infraestrutura de rede resiliente (ex: rede mesh) e a implementação de uma lógica de contingência no dispositivo IoT, como o armazenamento local de eventos ('store-and-forward') e a emissão de alertas sonoros locais caso a conexão com o backend seja perdida.

8.3 Viabilidade Técnica a Longo Prazo: Escalabilidade, Evolução e Manutenção

A sustentabilidade do sistema a longo prazo é assegurada pelas decisões arquiteturais e processuais tomadas desde a sua concepção. A escalabilidade é garantida nativamente pela arquitetura de microsserviços em conjunto com a orquestração do Kubernetes, permitindo que o sistema cresça de dezenas para milhares de usuários com ajustes de recursos, sem a necessidade de re-arquitetura. A evolução contínua é viabilizada pelo pipeline de DevSecOps e pelo baixo acoplamento entre os serviços, o que permite que novas funcionalidades, como um modelo de IA aprimorado, sejam implementadas e entregues de forma rápida e segura. Finalmente, a manutenção é simplificada através de um alto grau de automação nos testes, na análise de qualidade de código (SonarQube) e na observabilidade

do sistema em produção (Prometheus/Grafana). Este tripé, arquitetura, automação e observabilidade, garante que o sistema não apenas atenda aos requisitos atuais, mas que esteja preparado para crescer e se adaptar a futuros desafios.

Referências

- [1] Martin Fowler. Technical debt quadrant. Disponível em: <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>, 2009. Acesso em: 31 ago. 2025.
- [2] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. Bookman, 2000.
- [3] Jez Humble and David Farley. *Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável*. Alta Books, 2019.
- [4] Ulrich Hunkeler, Hong Linh Truong, and Andy Stanford-Clark. MQTT-S — A publish/subscribe protocol for Wireless Sensor Networks. In *2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE '08)*, 2008.
- [5] Gene Kim, Kevin Behr, and George Spafford. *Projeto Fênix: Um Romance sobre TI, DevOps e Sobre Ajudar sua Empresa a Vencer*. Alta Books, 2015.
- [6] Robert C. Martin. *Arquitetura Limpa: O Guia do Artesão para Estrutura e Design de Software*. Alta Books, 2019.
- [7] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2 edition, 2021.
- [8] Chris Richardson. *Microserviços Patterns: Com Exemplos em Java*. Novatec Editora, 2019.